

GURUKUL

Software Requirements Specification

Multi-Category Personal Growth Platform

Version: 1.0 | Status: Draft | Date: August 2024

1. EXECUTIVE SUMMARY

GURUKUL is a multi-category personal growth platform democratizing access to world-class coaching and personalized learning. Launches with Gym/Fitness, expanding to English, Cricket, Career, Meditation. CRITICAL: Architecture is NOT fitness-specific at platform core.

Key Differentiators:

- AI-driven personalization adapting to individual learning styles and goals
- Premium visual design blending Indian wisdom with modern technology (#D4AF37 gold, dark theme, cinematic imagery)
- Modular category architecture enabling rapid expansion (Gym → English → Cricket → Future)
- Holistic development approach (mind, body, soul) not siloed fitness
- Community-driven engagement with peer support and social accountability
- Progress gamification with daily streaks, achievements, and status visualization

MVP Scope (2 Weeks): Landing page with hero, goal selection, program browsing, registration/login flow, basic onboarding questionnaire.

2. PRODUCT VISION

"Ancient Wisdom. Modern Guidance. Limitless Potential."

GURUKUL bridges ancient ashram philosophy (holistic learning from master teachers) with cutting-edge AI. Named after Gurukul educational systems where students learned from gurus—reimagined for 2024 where your personal guru is AI, infinitely patient, available 24/7.

Positioning

- Premium positioning (NOT commodity fitness app like Peloton/Fittr)
- Premium visual & interaction design (avoid SaaS template look)
- AI-powered personal coaches available 24/7 across multiple domains
- Mission-driven transformation focus, not just transaction
- Community-first engagement (peer support > gamification)

3. PLATFORM ARCHITECTURE

Three-layer system: Shared Platform Services → Category Engine → Domain-Specific Features

LAYER 1: SHARED PLATFORM SERVICES

Core infrastructure shared across ALL categories (critical architectural principle):

- Authentication (JWT, email/password, OAuth2 infrastructure, MFA-ready)
- User Management (profiles, preferences, settings, privacy controls)
- Notifications (push, email, in-app SMS-ready)
- Payments & Subscriptions (Stripe integration, usage tracking)
- Analytics & Observability (PostHog, Sentry, OpenTelemetry, session replay)
- Search (full-text across programs, exercises, community content)
- Media Management (Cloudinary CDN, image optimization, video streaming)
- Community Foundation (profiles, verification, moderation tools)
- Admin Dashboard (multi-tenant, category-agnostic)
- Feature Flags (Unleash/LaunchDarkly for A/B testing)
- Content Management System (programs, exercises, lessons)
- AI Infrastructure (LLM abstraction, context management, token budgeting)

LAYER 2: CATEGORY ENGINE

Vertical-specific domains that inherit platform but maintain independent logic:

- Gym: Goals, Programs, Exercises, Workouts, Progress, 1RM tracking, AI Fitness Coach
- English: Lessons, Assessments, Speaking Practice, Grammar, Pronunciation API, AI English Coach
- Cricket: Drills, Video Analysis, Strategy, Form Coaching, Match Replays, AI Cricket Coach
- Future Categories: Extensible architecture (Career, Meditation, Leadership, etc.) without platform rewrite

LAYER 3: DOMAIN-SPECIFIC FEATURES

- Gym: Form videos, rep/set tracking, 1RM calculators, workout splits, equipment selection
- English: Pronunciation scoring, spaced repetition algorithm, speaking assessment, vocabulary tracking
- Cricket: Ball trajectory analysis, batting/bowling form correction, match strategy tools

4. TECHNOLOGY STACK

Production-grade MERN (MongoDB, Express, React, Node.js) selected for: unified JavaScript ecosystem, rapid iteration, proven scalability (Netflix, Uber, Airbnb, Stripe), strong job market, excellent developer experience.

FRONTEND

- React 18+ with Hooks and Concurrent Rendering, Vite build tool (3-4x faster than CRA)
- TypeScript (type safety, improved IDE, refactoring confidence)
- Tailwind CSS (utility-first, design tokens, rapid UI prototyping)
- Shadcn/UI (accessible, unstyled components built on Radix primitives)
- TanStack Query (server state, auto-caching, background sync, race condition prevention)
- Zustand (lightweight client state, simpler than Redux for app this size)
- React Router v6 (client-side routing, lazy code-splitting per route)
- React Hook Form + Zod (performant forms, schema validation, minimal re-renders)
- Framer Motion (smooth animations, gesture support, spring physics)
- Lucide React (lightweight icon library, consistent aesthetic)

BACKEND

- Node.js 20 LTS (long-term support, excellent async/await, mature ecosystem)
- Express.js 4.18+ (minimal, unopinionated, enables custom architecture without framework baggage)
- TypeScript (type safety at API boundaries, better error detection)
- MongoDB 6.0+ (document database, flexible schema, built-in replication)
- Mongoose 7+ (ODM with schema validation, middleware hooks, population)
- JWT (jsonwebtoken) for stateless authentication across distributed systems
- bcrypt (password hashing with adaptive salt rounds)
- Nodemailer (passwordless login, password reset, transactional email)
- Multer (multipart file upload handling)
- Winston (structured logging, multiple transports, log levels)
- Helmet (security headers: CSP, X-Frame-Options, X-Content-Type-Options)
- express-rate-limit (API abuse prevention, per-endpoint configuration)
- node-cron (scheduled jobs: daily notifications, progress calculations)

EXTERNAL SERVICES

- MongoDB Atlas (managed, backups, replication, no ops overhead)
- Stripe (payment processing, subscription management, invoice history)
- Anthropic Claude API (AI Coach conversations, content generation)
- Cloudinary (image optimization, responsive delivery, video streaming)
- Render or Railway (Node.js hosting, auto-scaling, CI/CD)
- Vercel (React hosting, global CDN, edge functions, preview deployments)
- PostHog (product analytics, feature flags, session replay)
- Sentry (error tracking, performance monitoring, alerts)
- Firebase Cloud Messaging (push notifications) or OneSignal

5. DATABASE ARCHITECTURE

MongoDB with hybrid embedding/referencing strategy optimized for read-heavy operations and real-time queries.

Core Collections

- Users: _id, email (unique), password_hash (bcrypt), full_name, avatar_url, status, created_at
Index: email (unique), created_at, status
- Profiles: user_id, bio, age, gender, timezone, language, fitness_level, health_conditions, preferences
Index: user_id (unique), fitness_level
- Categories: slug, name, description, icon_url, banner_url, status, order, config (category-specific)
Index: slug (unique), status
- Goals: user_id, category_id, goal_type, description, target_date, status, created_at
Index: user_id, category_id, status
- Programs: category_id, goal_ids (array), title, description, difficulty, duration_weeks, curriculum (embedded)
Index: category_id, goal_ids, difficulty
- Exercises/Lessons: category_id, name, description, media_url, duration, difficulty, target_area
Index: category_id, difficulty, target
- Workouts/Sessions: program_id, sequence_number, title, exercises (array), duration_minutes
Index: program_id, sequence_number
- Progress: user_id, program_id, workout_id, date_completed, duration_seconds, rating
Index: (user_id, program_id, date_completed) compound for efficient queries
- AIConversations: user_id, category_id, messages (embedded), context (user profile summary)
Index: user_id, category_id, created_at
- Subscriptions: user_id, plan_tier (free/basic/premium), stripe_subscription_id, status, period dates
Index: user_id, status
- Notifications: user_id, title, body, type, sent_at, read_at (TTL: 30 days)
Index: user_id, sent_at
- CommunityPosts: user_id, category_id, content, media_urls, likes_count, created_at
Index: user_id, category_id, created_at, likes_count

- AnalyticsEvents: event_name, user_id, session_id, properties (JSON), timestamp (TTL: 90 days)
Index: event_name, user_id, timestamp

6. API ARCHITECTURE

RESTful API (Base: <https://api.gurukul.app/api/v1>) with JWT auth, standardized errors

Authentication

- **POST /auth/register:** Register with email/password, returns access + refresh tokens
- **POST /auth/login:** Authenticate, returns tokens
- **POST /auth/refresh:** Refresh access token using refresh token
- **POST /auth/logout:** Invalidate tokens
- **POST /auth/password-reset:** Initiate password reset flow

Programs & Enrollment

- **GET /programs:** List, filter by category/goal/difficulty, search, paginate
- **GET /programs/{id}:** Program details with curriculum breakdown
- **POST /programs/{id}/enroll:** Enroll authenticated user in program
- **GET /categories:** List all active categories (public endpoint)
- **GET /categories/{slug}:** Category details, featured content

Progress & Goals

- **POST /progress/session:** Log workout/lesson completion
- **GET /progress:** User stats, graphs, streaks, achievement badges
- **POST /goals:** Create new goal
- **GET /goals:** List user's goals, filter by category
- **PUT /goals/{id}:** Update goal (description, target date)

AI & Community

- **POST /ai/chat:** Send message to AI Coach, receive reply + suggestions
- **GET /ai/conversations:** List user's chat conversations
- **POST /community/posts:** Create community post
- **GET /community/feed:** Get feed (followed users + trending)
- **POST /subscriptions/checkout:** Initiate Stripe checkout session

7. FRONTEND ARCHITECTURE

Feature-based organization by domain (auth, gym, user, community) enables independent development and reusability. Each feature is self-contained with components, hooks, services, state, types.

Folder Structure: src/pages/, src/features/{auth,gym,user,community}, src/components/, src/hooks/, src/services/, src/api/, src/store/, src/types/, src/utils/, src/constants/

State Management

Zustand (client state): user auth, preferences, UI modals, filters

TanStack Query (server state): programs, progress, posts, auto-sync with API

Separation ensures: UI ← Zustand ← TanStack Query ← API

8. BACKEND ARCHITECTURE

Layered: Controllers → Services → Repositories → MongoDB

Folder Structure: src/config/, src/controllers/, src/services/, src/repositories/, src/models/, src/routes/, src/middleware/, src/validators/, src/utils/, src/types/, src/jobs/

9. SECURITY ARCHITECTURE

Authentication

- JWT: 15-minute access tokens, 7-day refresh tokens in HTTP-only secure cookies
- Passwords: bcrypt with 12 salt rounds, salted hashing
- Email verification required on signup (prevents spam)
- Password reset: Email-based with time-limited tokens (24 hours)
- OAuth2 infrastructure ready: Google, GitHub auth scaffolding complete

Authorization (RBAC)

- Roles: user, moderator, admin, category_coach
- Resource-level access: Users can only access their own data
- Admin operations: All changes logged in audit trail with timestamp + actor
- API scopes: Tokens can be restricted to specific permission sets

Data Protection & API Security

- HTTPS enforced (TLS 1.3), all traffic encrypted
- Secrets in environment variables, never in code
- Sensitive data (passwords, payment tokens) encrypted at rest
- Rate limiting: 100 requests/hour per IP (tuned per endpoint)
- CORS: Restricted to frontend domain, credentials: include
- Input validation: Joi/Zod schemas on all endpoints
- Helmet: Security headers (X-Frame-Options, CSP, X-Content-Type-Options)
- XSS prevention: Input escaping, output encoding, CSP policies

10. INFRASTRUCTURE & DEPLOYMENT

Stack

Frontend: Vercel (automatic deployments, global CDN, edge functions, analytics)

Backend: Render or Railway (Docker, auto-scaling, CI/CD, environment management)

Database: MongoDB Atlas (managed service, backups, replication, encryption)

Media: Cloudinary (image optimization, responsive delivery, video streaming)

Monitoring: PostHog (product analytics, flags), Sentry (errors), OpenTelemetry (tracing)

Deployment Pipeline

1. Feature branch → PR preview (Vercel)
2. Merge to main → Staging deployment (Render)
3. QA testing on staging
4. Git tag (v1.2.3) → Production deployment (migrations, health checks, monitoring)

11. MVP SCOPE (2 WEEKS)

MUST HAVE

Landing Page:

- Responsive design (mobile-first, tablet, desktop)
- Hero section: tagline, CTA ("Start Your Journey")
- Goal selection: visual cards (strength, muscle, weight loss, etc.)
- Program discovery: cards with filters, search
- Why Gurukul: value prop, brand positioning
- How It Works: 4-5 step process
- AI Coach preview: feature highlight
- Progress visualization: example dashboard
- Testimonials: 3-5 social proof items
- FAQ: 5-10 common questions
- Footer: links, social, privacy

Authentication:

- User registration (email, password, confirm password)
- Email verification (link-based)
- Login with remember me option
- Logout functionality
- Password reset (email-based token flow)
- JWT token management + refresh token rotation

Onboarding:

- Goal selection component (fitness goals)
- Fitness level assessment (beginner/intermediate/advanced)
- Equipment availability (home, gym, no equipment)
- Time availability per week (3-4, 4-5, 5+ days)
- Personalized program recommendations based on answers
- Program preview before enrollment

SHOULD HAVE (Phase 1.5)

- Workout demonstration videos
- User profile management (edit name, avatar, bio)
- Basic AI Coach chats (limited conversations)
- Push notifications (enrollment confirmations, daily reminders)
- Email notifications (weekly digest)
- Stripe payment integration (basic checkout)

- Free vs. premium feature gating

NOT IN MVP

- English, Cricket, other categories
- Advanced AI Coach with personalized coaching plans
- Community features (posts, comments, challenges)
- Live coaching sessions
- Native iOS/Android app
- AR form correction
- Wearable integrations (Apple Health, Google Fit)
- Admin panel (full CMS)

12. 14-WEEK DEVELOPMENT ROADMAP

- **Weeks 1-2: Landing Page + Infrastructure** — Responsive landing, design system, backend scaffold, deployment pipeline. Deliverable: Live landing page.
- **Weeks 3-4: Auth + Onboarding** — Registration, login, email verification, password reset, questionnaire, recommendations. Deliverable: Users sign up and onboard.
- **Weeks 5-7: Programs & Workouts** — Program CRUD, exercise CRUD, program discovery, enrollment. Deliverable: Users browse and enroll.
- **Weeks 8-9: Progress Tracking** — Session logging, visualization, dashboard, streaks. Deliverable: Users track progress.
- **Week 10: AI Coach** — Claude API, chat interface, history, context. Deliverable: Users chat with coach.
- **Week 11: Payments** — Stripe integration, checkout, subscriptions. Deliverable: Users upgrade to paid.
- **Weeks 12-13: Polish & Optimization** — Performance, SEO, accessibility, mobile. Deliverable: Production-ready app.
- **Week 14: QA & Launch** — Final testing, security audit, monitoring. Deliverable: Live product.

13. DESIGN SYSTEM & VISUAL IDENTITY

Color Palette

- **Primary Gold (#D4AF37):** CTAs, accents, highlights, brand color
- **Dark Background (#1a1a2e):** Main background
- **Dark Secondary (#16213e):** Cards, sections
- **Text Primary (FFFFFF):** Main text on dark
- **Text Secondary (#A0A0A0):** Muted text, secondary info
- **Success (#10B981):** Progress, achievements, positive actions
- **Warning (#F59E0B):** Caution, alerts
- **Error (#EF4444):** Errors, destructive actions

Typography

Font Family: Inter, Poppins (modern, clean sans-serif)

H1: 56px bold | H2: 40px bold | H3: 28px semibold

Body: 16px normal (line-height 1.6) | Small: 14px | Caption: 12px opacity 0.7

Spacing (8px base)

xs: 4px | sm: 8px | md: 16px | lg: 24px | xl: 32px | 2xl: 48px | 3xl: 64px

14. RISK REGISTER

- **Scope Creep (HIGH prob, HIGH impact):** Strict MVP definition, weekly scope reviews, feature flags
- **AI Coach Token Costs (MEDIUM prob, HIGH impact):** Token budgets, caching, prompt optimization
- **Media/Storage Costs (MEDIUM prob, MEDIUM impact):** Cloudinary optimization, CDN caching, lazy loading
- **Category Expansion Complexity (MEDIUM prob, HIGH impact):** Domain-driven design, clear API contracts, docs
- **Performance at Scale (MEDIUM prob, HIGH impact):** DB indexing, query optimization, load testing

15. FINAL RECOMMENDATIONS

1. **1.** Prioritize MVP clarity: Landing page only weeks 1-2. Strict scope prevents delays.
2. **2.** Invest in design system early: Consistent visual drives premium positioning.
3. **3.** Separate concerns from day 1: Category logic separate from platform core.
4. **4.** Automate deployment: CI/CD, testing, monitoring configured at launch.
5. **5.** Plan AI costs aggressively: Token budgets, caching, optimization as users grow.
6. **6.** Document architecture decisions: ADRs save design debates and onboarding.
7. **7.** Build observability from day 1: Sentry, PostHog, logging at launch.
8. **8.** Validate on Gym first: Prove model before launching English/Cricket.
9. **9.** Focus on onboarding: Personalization funnel drives engagement.
10. **10.** Plan retention: Daily streaks, gamification, community by Phase 2.

GURUKUL PLATFORM — COMPLETE SRS

This specification provides the complete technical blueprint for implementing GURUKUL as a scalable multi-category personal growth platform. All teams can reference this throughout development.